

Task 5 — Wireshark Network Traffic Capture and Analysis

Author: Rohit Madhav Sabale

Executive Summary

This report documents the process and findings of capturing live network packets using Wireshark on Kali Linux. The goal was to observe and analyze common network protocols, identify communication patterns, and understand data flow. Protocols such as DNS, HTTP, HTTPS, TCP, and ICMP were identified during the capture session.

Environment & Tools

System: Kali Linux (latest version)

Tool: Wireshark

Interface Used: wlan0 (Wi-Fi)

Additional Tools: ping, curl commands to generate traffic

Methodology

- Updated system and installed Wireshark:
`#sudo apt update && sudo apt install wireshark -y`
- Started Wireshark and selected the wlan0 interface for live packet capture.
- Generated network traffic by visiting websites and running ping/curl commands:
ping google.com
curl http://example.com
- Captured packets for approximately one minute and saved them as task5_capture.pcap.
- Applied Wireshark filters (http, dns, icmp, tls) to analyze protocol details.

Screenshots

Below are screenshots captured during the experiment:

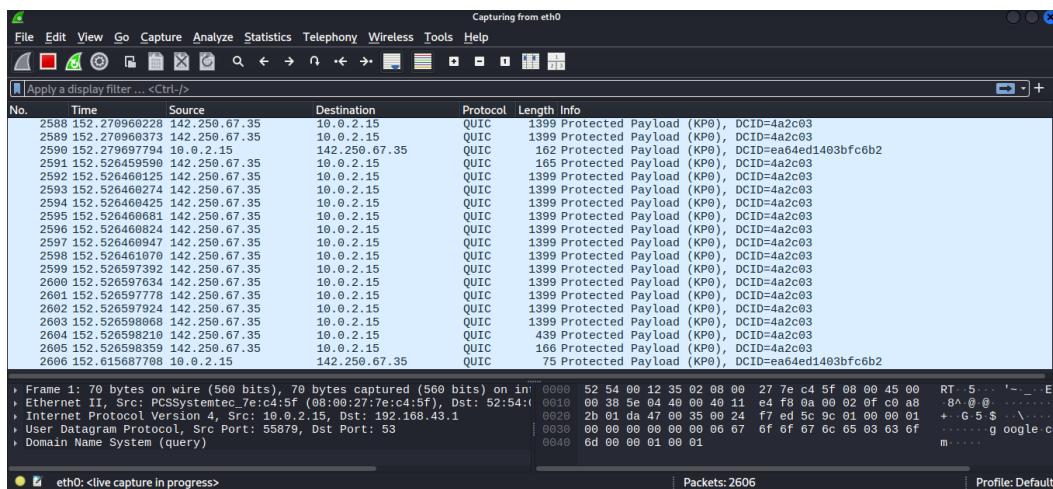


Figure: Wireshark capture and protocol analysis.

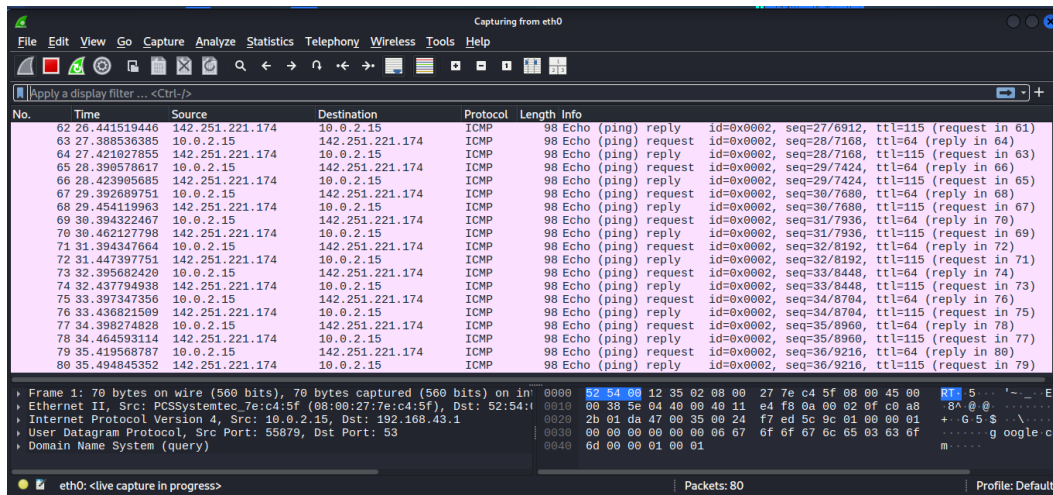


Figure: Wireshark capture and protocol analysis.

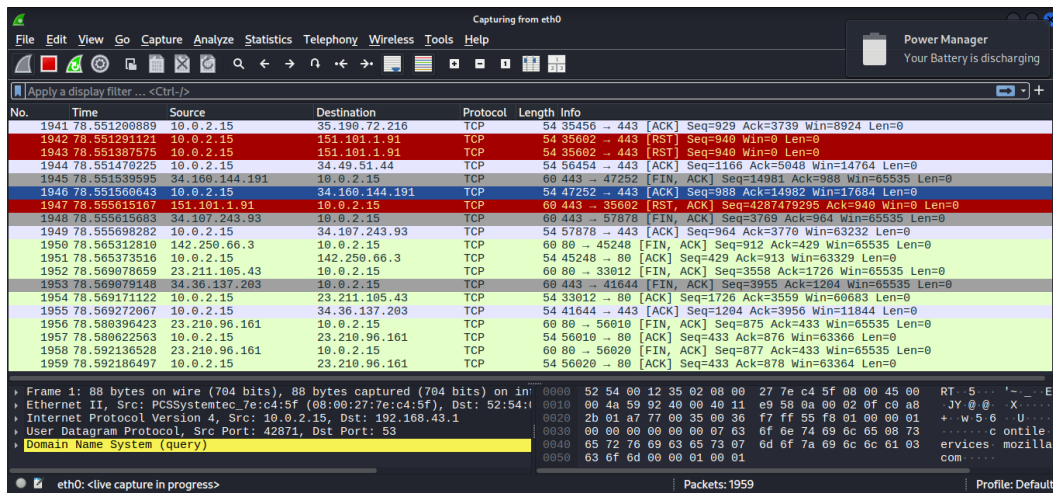


Figure: Wireshark capture and protocol analysis.

Protocols Identified

During analysis, several key network protocols were observed. Each protocol has a distinct purpose in the communication process:

- DNS: Domain Name System — Resolves domain names to IP addresses (Port 53, UDP).
- HTTP: Hypertext Transfer Protocol — Transfers unencrypted web data (Port 80, TCP).
- HTTPS: Hypertext Transfer Protocol Secure — Encrypted communication using TLS (Port 443, TCP).
- ICMP: Internet Control Message Protocol — Used for diagnostics like ping (no port).
- TCP: Transmission Control Protocol — Reliable connection-oriented communication.
- UDP: User Datagram Protocol — Fast, connectionless communication used in DNS and streaming.

Observations

- DNS queries were observed to external DNS servers (8.8.8.8 - Google DNS).
- HTTP requests were sent to example.com with visible GET headers.
- ICMP packets indicated successful connectivity with echo replies from google.com.
- TLS handshake packets were visible during HTTPS traffic capture.
- Multiple TCP SYN, SYN-ACK, and ACK packets were exchanged, confirming reliable connections.

Conclusion

This practical exercise helped in understanding real-time packet flow and behavior of different protocols. Wireshark provided deep visibility into each protocol's structure, enabling effective traffic analysis. This forms the foundation for advanced network monitoring and cybersecurity investigations.

*****THANK YOU*****